

Relatório de Refatoração - Bad Smells em JavaScript

- **Disciplina:** Teste de Software
- **Trabalho:** Detecção e Refatoração de Bad Smells em JavaScript
- **Aluno:** Thiago Borges Laass
- **Matrícula:** 836095
- **Repositório:** <https://github.com/ThiagoLaass/bad-smells-js-refactoring>.

Resumo

- **Objetivo:** identificar Bad Smells no código original, usar ESLint + `eslint-plugin-sonarjs` para detecção, refatorar em uma cópia segura (`src/ReportGenerator.refactored.js`) e validar com a suíte de testes.
- **Resultado:** criado `src/ReportGenerator.refactored.js` e `tests/ReportGenerator.refactored.test.js` ; ambos os conjuntos de testes (original e refatorado) passam; ESLint deixou de reportar os problemas principais no arquivo refatorado.

1. Análise Manual (Smells identificados)

- **Método Longo / Alta Complexidade Ciclomática:** a função `generateReport` em [src/ReportGenerator.js](#) concentra cabeçalho, corpo e rodapé, com muitos ramos condicionais aninhados (tipo de relatório × papel do usuário × regras de filtragem), tornando-a difícil de entender e manter.
- **Condicionais Aninhadas / Muitos If/Else:** lógica espalhada entre blocos `if (user.role === 'ADMIN')` e `if (reportType === 'CSV' | 'HTML')` , que causa duplicação e dificulta alteração futura.
- **Duplicação de Formatação:** geração de linhas CSV/HTML repetida em vários locais do loop, aumentando a chance de inconsistência ao alterar o formato.

Por que são problemáticos:

- Aumentam custo de modificação e introduzem risco de regressão.
- Dificultam a leitura e testes isolados de regras (ex.: prioridade de item, filtro por valor).

2. Configuração e Execução da Ferramenta

- Instalei dependências e configurei suporte a ESM para os testes (Babel / babel-jest) para poder executar Jest com arquivos `import` .
- Comando de análise executado (antes da refatoração):

```
npx eslint src/ --ext .js
```

Saída relevante do ESLint (antes):

```
src/ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5
allowed sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one
sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

- Interpretação: o plugin `eslint-plugin-sonarjs` identificou complexidade cognitiva alta e condição aninhada que pode ser simplificada.

3. Processo de Refatoração Objetivo principal: reduzir a complexidade cognitiva e eliminar duplicação, preservando o comportamento (garantido por testes).

Passos aplicados:

- Criar uma cópia segura: `src/ReportGenerator.refactored.js` (não alterar o original diretamente).
- Extrair responsabilidades: separar construção de `header`, `body` e `footer` em métodos próprios (`buildHeader`, `buildBody`, `buildFooter`).
- Extrair funções utilitárias: `getVisibleItems(user, items)`, `sumValues(items)`, `formatCsvRow(item, user)`, `formatHtmlRow(item, user)`.
- Remover duplicação de formatação ao centralizar as templates de linha.

Trecho antes da refatoração:

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) { item.priority = true; }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;" : '';
      report += `|
|  |

```

Trecho refatorado:

```
getVisibleItems(user, items) {
  if (user.role === 'ADMIN') return items;
  return items.filter(item => item.value <= 500);
}

buildBody(reportType, user, items) {
  if (reportType === 'CSV') return items.map(i => this.formatCsvRow(i, user)).join('');
  if (reportType === 'HTML') return items.map(i => this.formatHtmlRow(i, user)).join('');
  return '';
}

formatHtmlRow(item, user) {
  const isPriority = user.role === 'ADMIN' && item.value > 1000;
  const style = isPriority ? ' style="font-weight:bold;" : '';
  return `
```

Efeito prático:

- A função pública `generateReport` ficou enxuta: monta header, body e footer chamando helpers.
- Redução direta da complexidade cognitiva por delegação.

4. Validação (Testes e ESLint)

- Testes executados:

```
npm test
```

- Resultado: todos os testes passaram para ambas as versões (original + refatorado).
 - Arquivos de teste: [tests/ReportGenerator.test.js](#) e [tests/ReportGenerator.refactored.test.js](#).
- ESLint executado após refatoração no arquivo refatorado:

```
npx eslint src/ReportGenerator.refactored.js --ext .js
```

- Saída: nenhum erro reportado para `src/ReportGenerator.refactored.js`.

5. Antes / Depois

- Arquivo original: [src/ReportGenerator.js](#)
- Arquivo refatorado: [src/ReportGenerator.refactored.js](#)
- Testes originais: [tests/ReportGenerator.test.js](#)
- Testes refatorados: [tests/ReportGenerator.refactored.test.js](#)

6. Conclusão

- Refatoração segura requer uma boa suíte de testes: neste exercício, os testes permitiram reestruturar o código sem alterar comportamento observável.